

SECURITY

SPIS TREŚCI

Spis treści	1
Cel zajęć.....	1
Rozpoczęcie	1
Uwaga	1
Zabezpieczenie dostępu do akcji i widoków	2
Proste logowanie Basic Auth	7
Użytkownicy bazodanowi.....	10
Formularz logowania.....	12
Wylogowywanie	15
Hierarchia ról.....	18
Commit projektu do GIT.....	20
Podsumowanie.....	20

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie umiejętności implementacji i konfiguracji logowania i wylogowywania użytkowników oraz ograniczania dostępu do akcji.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie security is_granted w Twig i atrybutach kontrolerów.

Wejściówka?

UWAGA

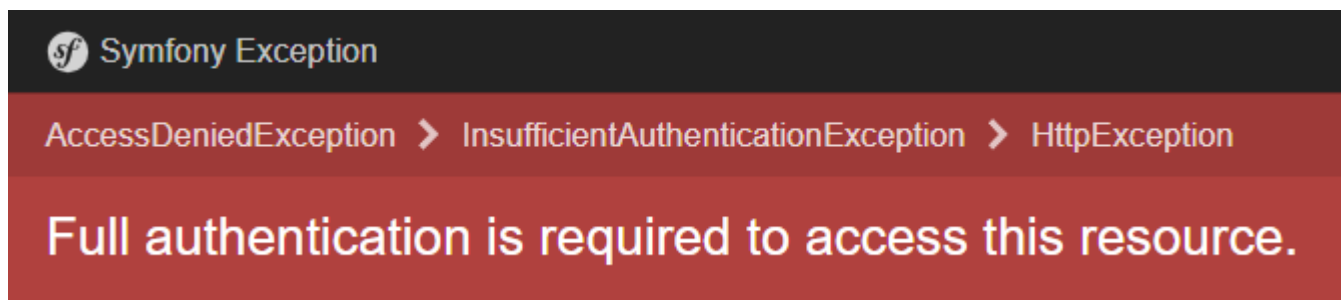
Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do Plik -> Informacje -> Właściwości -> Właściwości zaawansowane -> Niestandardowe i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub Ctrl+A -> F9.

ZABEZPIECZENIE DOSTĘPU DO AKCJI I WIDOKÓW

Obecnie dostęp do wszystkich akcji w systemie jest publicznie otwarty. W tej sekcji zabezpieczymy dostęp. Otwórz plik `src/Controller/LocationController.php` i edytuj akcję `new`:

```
#[Route('/new', name: 'app_location_new', methods: ['GET', 'POST'])]
#[IsGranted('ROLE_LOCATION_NEW')]
public function new(Request $request, EntityManagerInterface $entityManager): Response
{
    $location = new Location();
    $form = $this->createForm(LocationType::class, $location, [
        'validation_groups' => 'create',
    ]);
    $form->handleRequest($request);
```

Odwiedź teraz w przeglądarce stronę `http://pogodynka.localhost:51641/location`. Powinna wyświetlić się lista lokalizacji. Następnie kliknij w przycisk tworzenia nowej lokalizacji. Powinien wyświetlić się błąd:



Dlaczego lista lokalizacji wciąż działa, a akcja tworzenia lokalizacji wyrzuca wyjątek?

Brak uprawnień do tworzenia nowych lokalizacji

...maksymalnie 3 zdania omówienia...

Teraz zmodyfikuj widok listy lokalizacji w pliku `templates/location/index.html.twig`:

```
--- a/templates/location/index.html.twig
+++ b/templates/location/index.html.twig
@@ -38,5 +38,7 @@
     </tbody>
 </table>

-    <a href="{{ path('app_location_new') }}">Create new</a>
+    {% if is_granted('ROLE_LOCATION_NEW') %}
+    <a href="{{ path('app_location_new') }}">Create new</a>
+    {% endif %}
+{% endblock %}
```

Ponownie wejdź w przeglądarce na listę lokalizacji. Przycisk tworzenia nowej lokalizacji powinien teraz zniknąć.

Dodaj wymuszanie posiadania przez użytkownika roli dla wszystkich akcji kontrolerów lokalizacji i pomiarów. Zabronione jest korzystanie z roli typu `ROLE_ADMIN`. Zamiast tego utwórz dla każdej akcji w każdym kontrolerze osobną rolę odnoszącą się do czynności, w postaci `ROLE_<kontroler>_<akcja>`, np. `ROLE_LOCATION_INDEX`.

Poniżej wstaw zrzuty ekranu kodu atrybutów i sygnatury każdej akcji każdego kontrolera:

`LocationController::new():`

Artur Karczmarczyk

```
#[Route('/new', name: 'app_location_new', methods: ['GET', 'POST'])]
#[IsGranted('ROLE_LOCATION_NEW')]
public function new(Request $request, EntityManagerInterface $entityManager): Response
```

LocationController::index():

```
#[Route('/', name: 'app_location_index', methods: ['GET'])]
#[IsGranted('ROLE_LOCATION_INDEX')]
public function index(LocationRepository $locationRepository): Response
{
    return $this->render('location/index.html.twig', [
        'locations' => $locationRepository->findAll(),
    ]);
}
```

LocationController::show():

```
#[Route('/{id}', name: 'app_location_show', methods: ['GET'])]
#[IsGranted('ROLE_LOCATION_SHOW')]
public function show(Location $location): Response
{
    return $this->render('location/show.html.twig', [
        'location' => $location,
    ]);
}
```

LocationController::edit():

```
#[Route('/{id}/edit', name: 'app_location_edit', methods: ['GET', 'POST'])]
#[IsGranted('ROLE_LOCATION_EDIT')]
public function edit(Request $request, Location $location, EntityManagerInterface $entityManager): Response
{
    $form = $this->createForm(LocationType::class, $location);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $entityManager->flush();

        return $this->redirectToRoute('app_location_index', [], Response::HTTP_SEE_OTHER);
    }

    return $this->render('location/edit.html.twig', [
        'location' => $location,
        'form' => $form,
    ]);
}
```

LocationController::delete():

```
#[Route('/{id}', name: 'app_location_delete', methods: ['POST'])]
#[IsGranted('ROLE_LOCATION_DELETE')]
public function delete(Request $request, Location $location, EntityManagerInterface $entityManager): Response
{
    if ($this->isCsrfTokenValid('delete', $location->getId(), $request->request->get('_token'))) {
        $entityManager->remove($location);
        $entityManager->flush();
    }

    return $this->redirectToRoute('app_location_index', [], Response::HTTP_SEE_OTHER);
}
```

MeasurementController::new():

```
#[Route('/new', name: 'app_measurement_new', methods: ['GET', 'POST'])]
#[IsGranted('ROLE_MEASUREMENT_NEW')]
public function new(Request $request, EntityManagerInterface $entityManager): Response
{
    $measurement = new Measurement();
    $form = $this->createForm(MeasurementType::class, $measurement);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $entityManager->persist($measurement);
        $entityManager->flush();

        return $this->redirectToRoute('app_measurement_index', [], Response::HTTP_SEE_OTHER);
    }

    return $this->render('measurement/new.html.twig', [
        'measurement' => $measurement,
        'form' => $form,
    ]);
}
```

MeasurementController::index():

```
#[Route('/', name: 'app_measurement_index', methods: ['GET'])]
#[IsGranted('ROLE_MEASUREMENT_INDEX')]
public function index(MeasurementRepository $measurementRepository): Response
{
    return $this->render('measurement/index.html.twig', [
        'measurements' => $measurementRepository->findAll(),
    ]);
}
```

MeasurementController::show():

```
#[Route('/{id}', name: 'app_measurement_show', methods: ['GET'])]
#[IsGranted('ROLE_MEASUREMENTR_SHOW')]
public function show(Measurement $measurement): Response
{
    return $this->render('measurement/show.html.twig', [
        'measurement' => $measurement,
    ]);
}
```

MeasurementController::edit():

```
#[Route('/{id}/edit', name: 'app_measurement_edit', methods: ['GET', 'POST'])]
#[IsGranted('ROLE_MEASUREMENTR_EDIT')]
public function edit(Request $request, Measurement $measurement, EntityManagerInterface $entityManager): Response
{
    $form = $this->createForm(MeasurementType::class, $measurement);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $entityManager->flush();

        return $this->redirectToRoute('app_measurement_index', [], Response::HTTP_SEE_OTHER);
    }

    return $this->render('measurement/edit.html.twig', [
        'measurement' => $measurement,
        'form' => $form,
    ]);
}
```

MeasurementController::delete():

```
#[Route('/{id}', name: 'app_measurement_delete', methods: ['POST'])]
#[IsGranted('ROLE_MEASUREMENTR_DELETE')]
public function delete(Request $request, Measurement $measurement, EntityManagerInterface $entityManager): Response
{
    if ($this->isCsrfTokenValid('delete.'.$measurement->getId(), $request->request->get('_token'))) {
        $entityManager->remove($measurement);
        $entityManager->flush();
    }

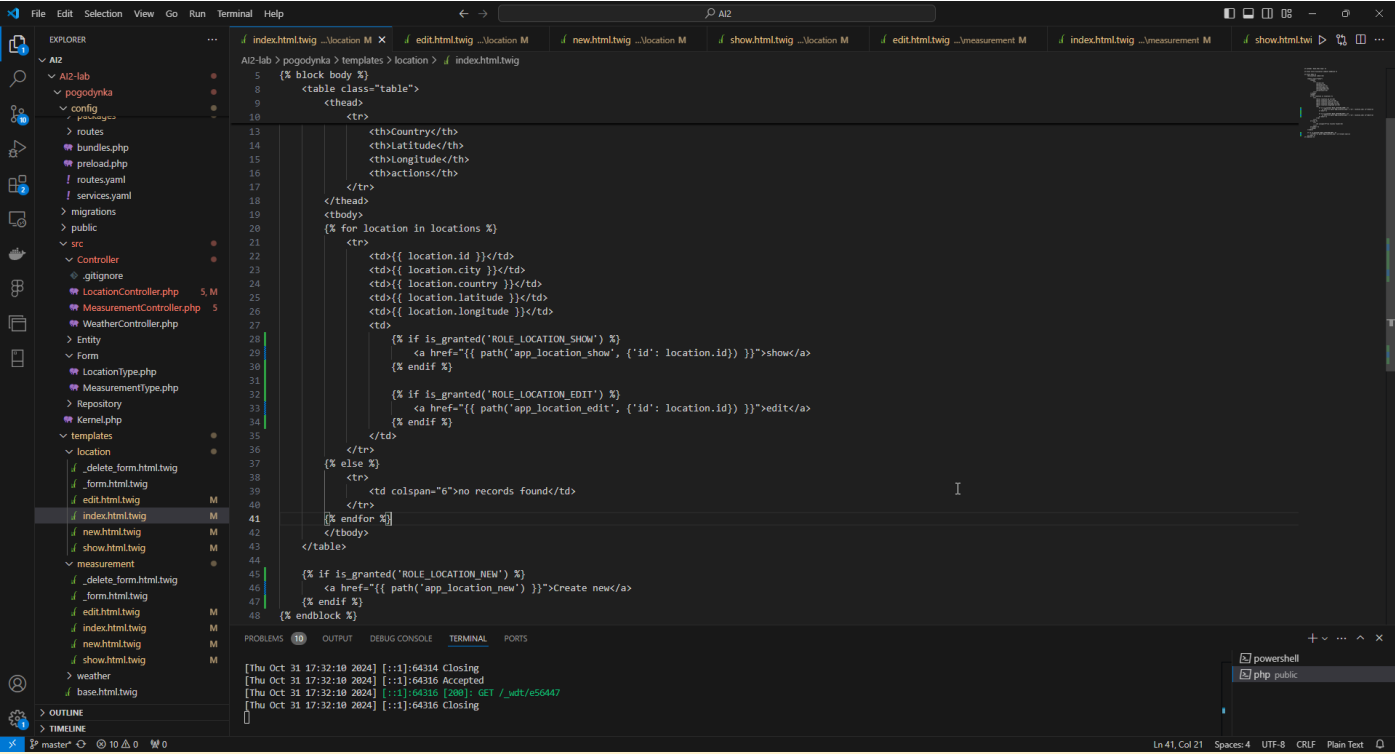
    return $this->redirectToRoute('app_measurement_index', [], Response::HTTP_SEE_OTHER);
}
```

Punkty:	0	1
---------	---	---

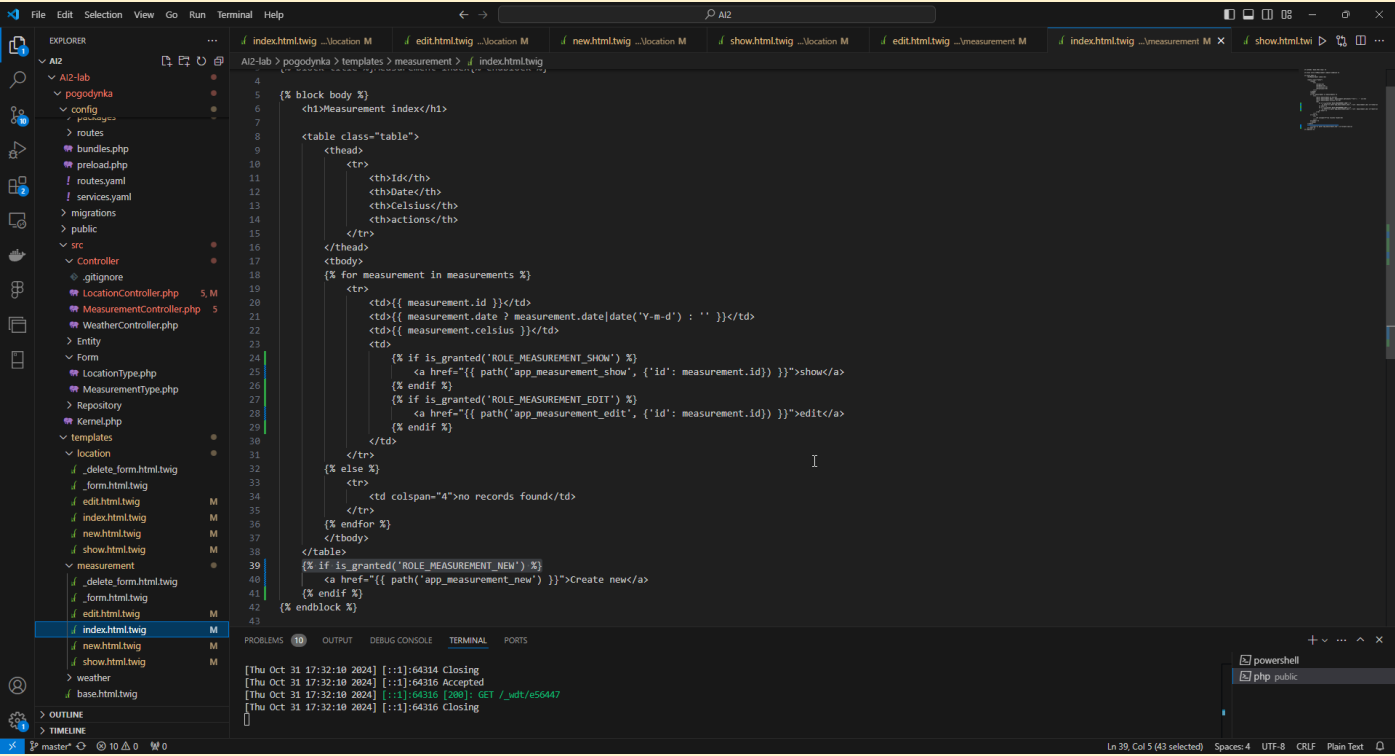
Dodaj weryfikację ról w szablonach TWIG na liście lokalizacji i liście pomiarów.

Wstaw zrzut ekranu kodu pliku `templates/location/index.html.twig`:

AI2 LAB E – Zielaskowski Dawid – Wersja 1



Wstaw rzzut ekranu kodu pliku templates/measurement/index.html.twig:



Punkty:	0	1
---------	---	---

PROSTE LOGOWANIE BASIC AUTH

W tej sekcji dodamy możliwość logowania użytkownika poprzez Basic Auth (login i hasło podawane bezpośrednio w adresie URL – `http://login:pass@pogodynka.localhost:51641` – lub w wyświetlonym przez przeglądarkę okienku).

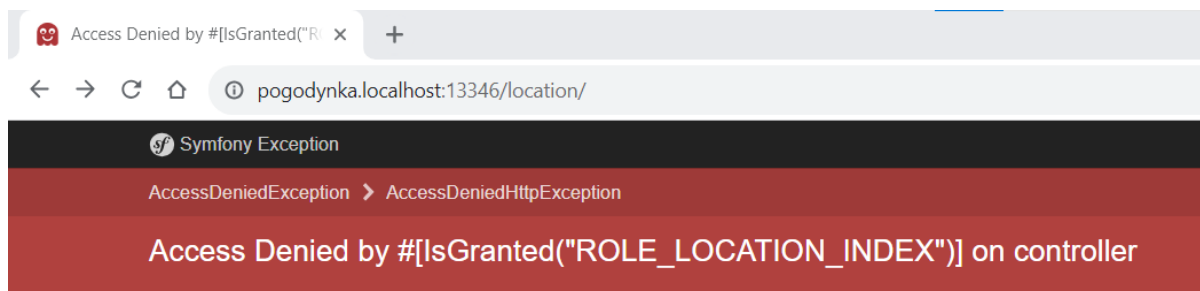
Modyfikuj plik `config/packages/security.yaml`:

```
--- a/config/packages/security.yaml
+++ b/config/packages/security.yaml
@@ -1,10 +1,17 @@
 security:
     # https://symfony.com/doc/current/security.html#registering-the-user-hashing-passwords
     password_hashers:
 -         Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
 +         Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'plaintext'
     # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
     providers:
 -         users_in_memory: { memory: null }
 +         users_in_memory:
 +             memory:
 +                 users:
 +                     admin:
 +                         password: '12345678'
 +                         roles:
 +                             - 'ROLE_ADMIN'
 +                             - 'ROLE_USER'
     firewalls:
         dev:
             pattern: ^/(_(profiler|wdt)|css|images|js)/
@@ -12,6 +19,8 @@ security:
     main:
         lazy: true
         provider: users_in_memory
 +         http_basic:
 +             realm: 'My Secured Area'

     # activate different ways to authenticate
     # https://symfony.com/doc/current/security.html#the-firewall
@@ -22,8 +31,7 @@ security:
     # Easy way to control access for large sections of your site
     # Note: Only the *first* access control that matches will be used
     access_control:
 -         # - { path: ^/admin, roles: ROLE_ADMIN }
 -         # - { path: ^/profile, roles: ROLE_USER }
 +         - { path: ^/, roles: PUBLIC_ACCESS }
```

W efekcie po wejściu na stronę powinno wyświetlić się okienko logowania Basic Auth. **Zrób zrzut ekranu! Po wpisaniu poprawnego loginu i hasła to okienko się nie pojawi ponownie!**

Po poprawnym logowaniu wyświetlony zostanie jednak błąd braku dostępu:



Wynika to z faktu, że obecnie użytkownik admin ma tylko role `ROLE_ADMIN` i `ROLE_USER`, a wymagana jest rola `ROLE_LOCATION_INDEX`. W dalszej części tego laboratorium rozwiążemy ten problem. Na ten moment można tymczasowo zmienić wymaganą rolę w `LocationController::index()` na `ROLE_ADMIN`, w celu weryfikacji poprawności działania:

Location index

Id	City	Country	Latitude	Longitude	actions
1	Szczecin	PL	53.4289	14.553	weather
2	Police	PL	53.5521	14.5718	weather

200 @app_location_i 3830 ms 50.0 MiB 1 2 admin 20 ms 3 6.3.4

Zwróć uwagę, że chociaż lista się pojawiła, wszystkie linki zabezpieczone rolami `ROLE_<kontroler>_<akcja>` są ukryte.

Wstaw zrzut ekranu wyskakującego okienka Basic Auth na tle Twojej strony:

Zaloguj się, aby uzyskać dostęp do tej witryny

Domena http://pogodynka.localhost:51641/ wymaga autoryzacji

Nazwa użytkownika

Hasło

Zaloguj Anuluj

Punkty:	0	1
---------	---	---

W dalszej części wykorzystamy komendę `security:hash-password` do zaszyfrowania hasła przed umieszczeniem go w pliku konfiguracyjnym:

```
php .\bin\console security:hash-password
Symfony Password Hash Utility
```

```
=====
Type in your password to be hashed:
>
```

```
! [NOTE] The command will take care of generating a salt for you. Be aware that some hashers advise to let them
    generate their own salt. If you're using one of those hashers, please answer 'no' to the question below.
!     Provide the 'empty-salt' option in order to let the hasher handle the generation itself.
```

```
Confirm salt generation ? (yes/no) [yes]:
```

```
> no
```

Key	Value
Hasher used	Symfony\Component\PasswordHasher\Hasher\PlaintextPasswordHasher

```
[OK] Password hashing succeeded
```

Zwróć uwagę, że zaszyfrowane hasło to wciąż plaintext. Wynika to z konfiguracji w pliku security.yaml. Zmień w nim opcję plaintext z powrotem na auto i ponów hashowanie. Wygenerowany hash podstaw jako hasło użytkownika w pliku security.yaml.

Wstaw zrzut ekranu konfiguracji security.yaml z hasłem w postaci uzyskanego hashu:

```
File Edit Selection View Go Run Terminal Help
AI2-lab > config > packages > ! security.yaml
1 security:
2   # https://symfony.com/doc/current/security.html#registering-the-user-hashing-passwords
3   password_hashers:
4     Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface: 'auto'
5   # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
6   providers:
7     users_in_memory:
8       memory:
9         users:
10          admin:
11            password: '$2y$13$kaZdUKR15vX103xvI1wFR9nJP1V9egz6FHHGo8yeJe0bGPePaq'
12            roles:
13              - 'ROLE_ADMIN'
14              - 'ROLE_USER'
15   firewalls:
16     dev:
17       pattern: ^/(_(profiler|wdt)|css|images|js)/
18       security: false
19   main:
20     lazy: true
21     provider: users_in_memory
22     http_basic:
23       realm: 'My Secured Area'
24   # activate different ways to authenticate
25   # https://symfony.com/doc/current/security.html#the-firewall
```

```
PS C:\Users\dawid\Desktop\Studia\AI2-Lab\pogodynka> php .\bin\console security:hash-password
Symfony Password Hash Utility
Type in your password to be hashed:
>
=====
Key      Value
-----
Hasher used  Symfony\Component\PasswordHasher\Hasher\PlaintextPasswordHasher
Password hash $2y$13$kaZdUKR15vX103xvI1wFR9nJP1V9egz6FHHGo8yeJe0bGPePaq

! [NOTE] Self-salting hasher used: the hasher generated its own built-in salt.

[OK] Password hashing succeeded
```

Punkty:

0

1

UŻYTKOWNICY BAZODANOWI

W tej sekcji usuniemy użytkownika z `security.yaml` na rzecz użytkowników przechowywanych w bazie danych.

Wykonaj komendę `make:user`:

```
php .\bin\console make:user

The name of the security user class (e.g. User) [User]:
>

Do you want to store user data in the database (via Doctrine)? (yes/no) [yes]:
>

Enter a property name that will be the unique "display" name for the user (e.g. email, username, uuid) [email]:
> username

Will this app need to hash/check user passwords? Choose No if passwords are not needed or will be checked/hashed
by some other system (e.g. a single sign-on server).

Does this app need to hash/check user passwords? (yes/no) [yes]:
>

created: src/Entity/User.php
created: src/Repository/UserRepository.php
updated: src/Entity/User.php
updated: config/packages/security.yaml

Success!

Next Steps:
- Review your new App\Entity\User class.
- Use make:entity to add more fields to your User entity and then run make:migration.
- Create a way to authenticate! See https://symfony.com/doc/current/security.html
```

Zaktualizuj schemat bazy danych:

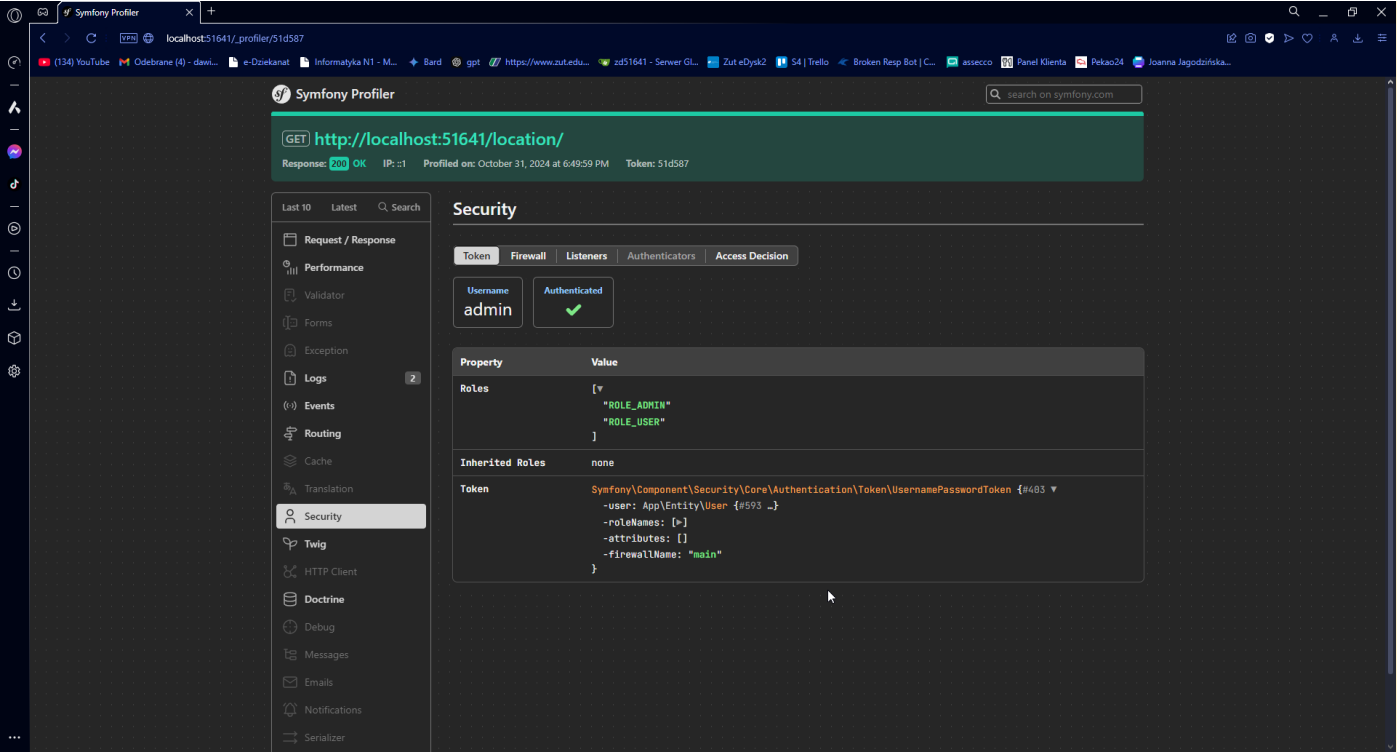
```
php bin\console doctrine:schema:update --dump-sql --force
```

Dodaj co najmniej jednego użytkownika bezpośrednio w bazie danych:

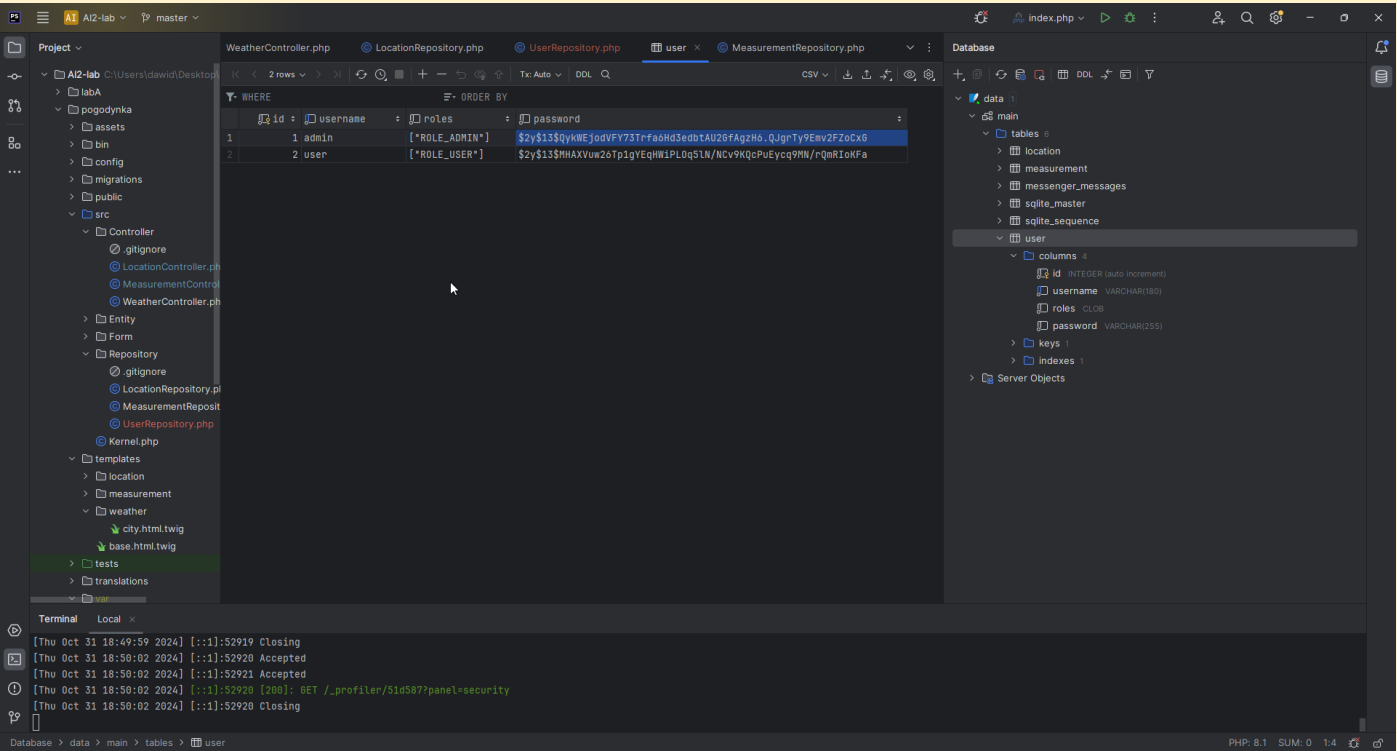
	id	username	roles	password
1	1	admin	["ROLE_ADMIN"]	\$2y\$13\$fhE1St0yxs0GnGc.ygWmt.c1A7...

Zaloguj się poprzez Basic Auth na użytkownika, którego dane zapisane są w bazie danych. W pasku profilera kliknij na nazwę użytkownika. Otworzy się panel Security.

Wstaw zrzut ekranu panelu Security z danymi zalogowanego użytkownika:



Wstaw zrzut ekranu podglądu tabeli bazy danych z danymi użytkownika zalogowanego na powyższym zrzucie ekranu:



Punkty:	0	1
---------	---	---

FORMULARZ LOGOWANIA

W tej sekcji wykorzystamy wbudowany w Symfony mechanizm `form_login` do obsługi formularza logowania. Wykorzystaj komendę `make:controller` do utworzenia kontrolera logowania:

```
php ./bin/console make:controller Login
created: src/Controller/LoginController.php
created: templates/login/index.html.twig
```

Success!

Next: Open your new controller class and add some pages!

Zaktualizuj ustawienia firewalla `main` w `security.yaml`. Zastąp logowanie `http_basic` przez `form_login`:

<code>main:</code>	16	16	<code>main:</code>
<code>lazy: true</code>	17	17	<code>lazy: true</code>
<code>provider: app_user_provider</code>	18	18	<code>provider: app_user_provider</code>
<code>http_basic:</code>	>> 19	19 ☐	<code>form_login:</code>
<code>realm: 'My Secured Area'</code>	20	20	<code>login_path: app_login</code>
	21	21	<code>check_path: app_login</code>

Teraz zmodyfikuj `LoginController`:

```
class LoginController extends AbstractController
{
    #[Route('/login', name: 'app_login')]
    public function index(AuthenticationUtils $authenticationUtils): Response
    {
        // get the login error if there is one
        $error = $authenticationUtils->getLastAuthenticationError();

        // last username entered by the user
        $lastUsername = $authenticationUtils->getLastUsername();

        return $this->render('login/index.html.twig', [
            'controller_name' => 'LoginController',
            'last_username' => $lastUsername,
            'error'           => $error,
        ]);
    }
}
```

Zmodyfikuj również szablon akcji logowania:

```

1  {% extends 'base.html.twig' %}
2
3  {% block title %}Login{% endblock %}
4
5  {% block body %}
6      {% if error %}
7          <div>{{ error.messageKey|trans(error.messageData, 'security') }}</div>
8      {% endif %}
9
10     <form action="{{ path('app_login') }}" method="post">
11         <label for="username">Username:</label>
12         <input type="text" id="username" name="_username" value="{{ last_username }}" />
13
14         <label for="password">Password:</label>
15         <input type="password" id="password" name="_password" />
16
17         {# If you want to control the URL the user is redirected to on success
18         <input type="hidden" name="_target_path" value="/admin" /> #}
19
20         <button type="submit">login</button>
21     </form>
22 {% endblock %}

```

Po wejściu na stronę z ograniczeniem dostępu wyświetli się teraz formularz logowania:

W App Cities Measurements

Username: Password:

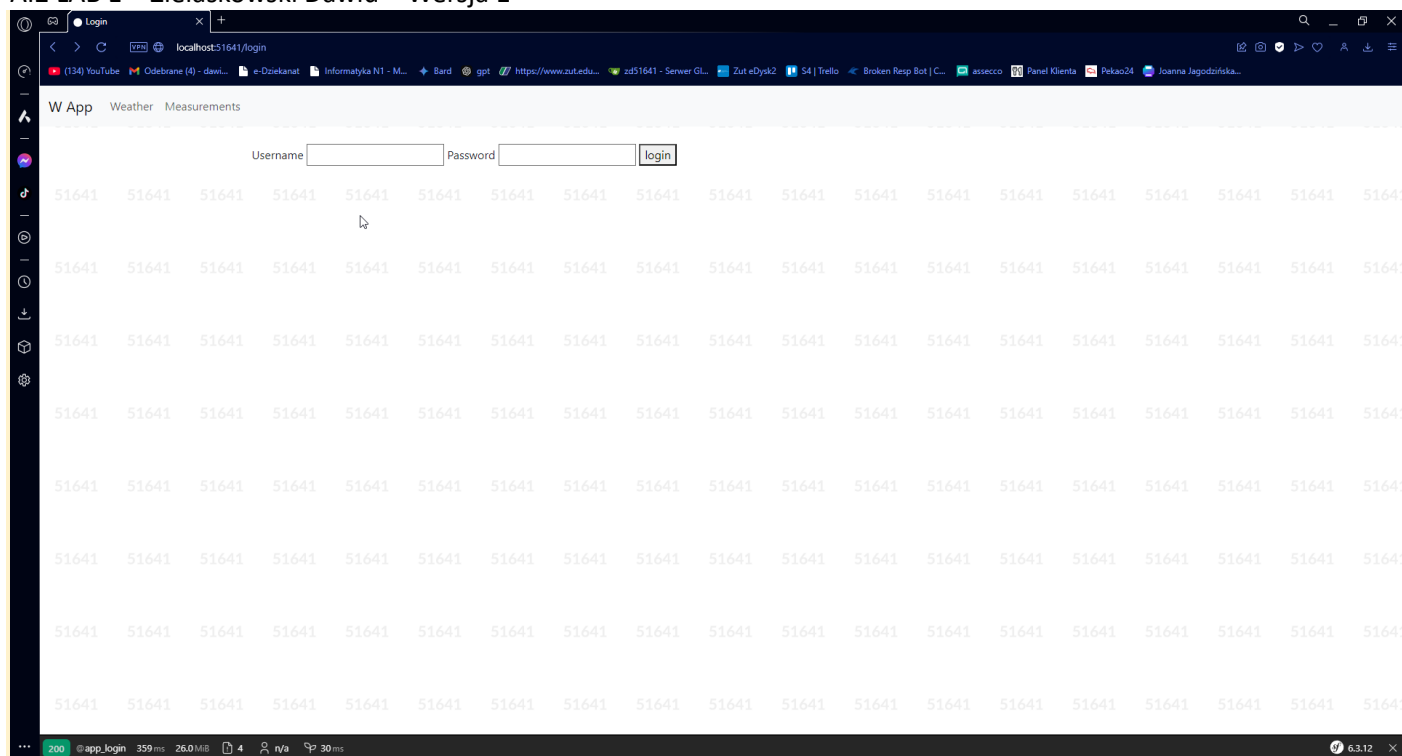
00000 00000 00000 00000 00000 00000 00000 00000 00000 00000

00000 00000 00000 00000 00000 00000 00000 00000 00000 00000

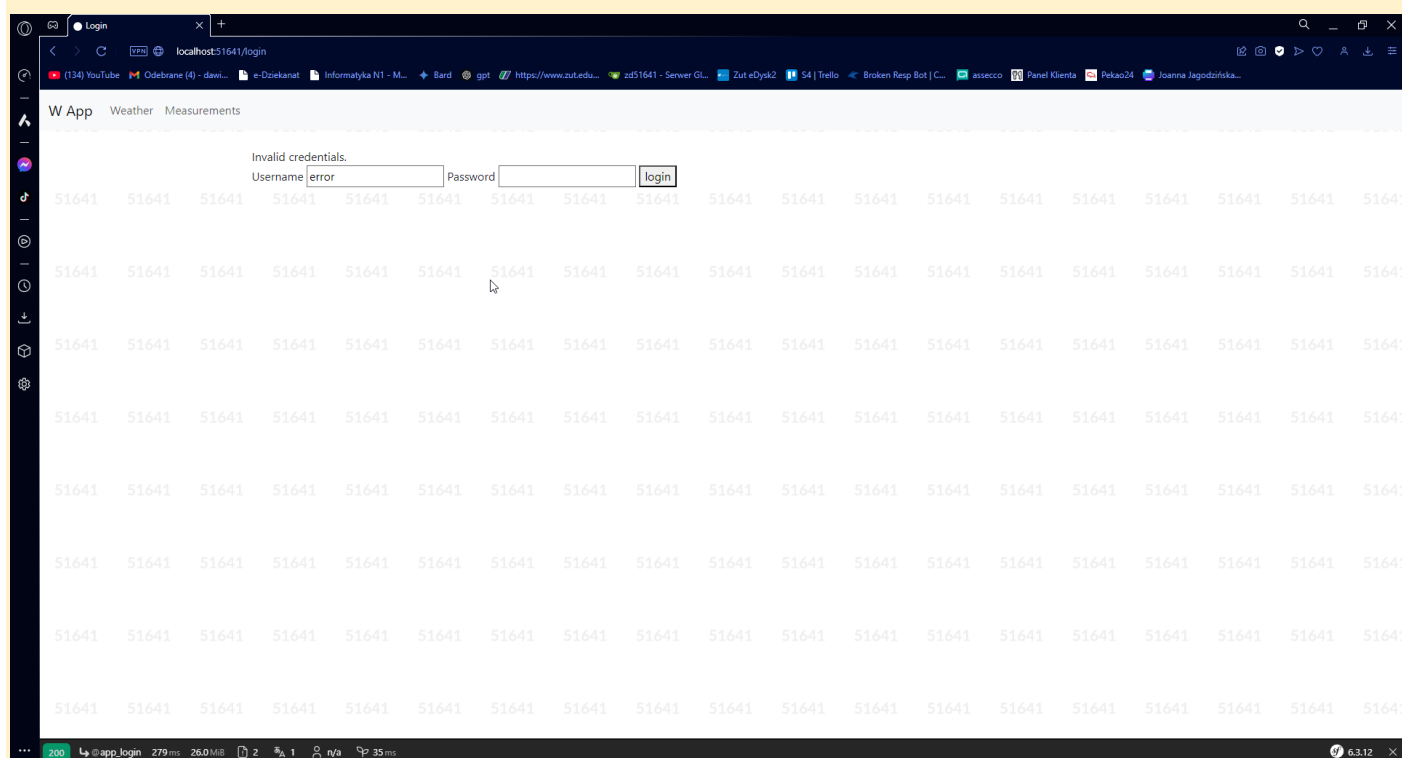
200 @app_login 284 ms 26.0 MiB admin 8 ms 1 6.3.4

Wstaw zrzut ekranu strony z formularzem logowania:

AI2 LAB E – Zielaskowski Dawid – Wersja 1



Podaj błędne dane logowania. Wstaw zrzut ekranu strony logowania z wyświetlonymi informacjami o błędach:



Zaloguj się na użytkownika, którego dane zapisane są w bazie danych. W pasku profilera kliknij na nazwę użytkownika. Otworzy się panel Security.

Wstaw zrzut ekranu panelu Security z danymi zalogowanego użytkownika:

Punkty:	0	1
---------	---	---

WYLOGOWYWANIE

Obecnie po zalogowaniu jedyną opcją wylogowania użytkownika jest skasowanie ciasteczka PHPSESSID. Dodanie logowania w Symfony jest proste. Wystarczy dodać nową ścieżkę do routes.yaml:

```
app_logout:
  path: /logout
  methods: GET
```

oraz zmodyfikować security.yaml:

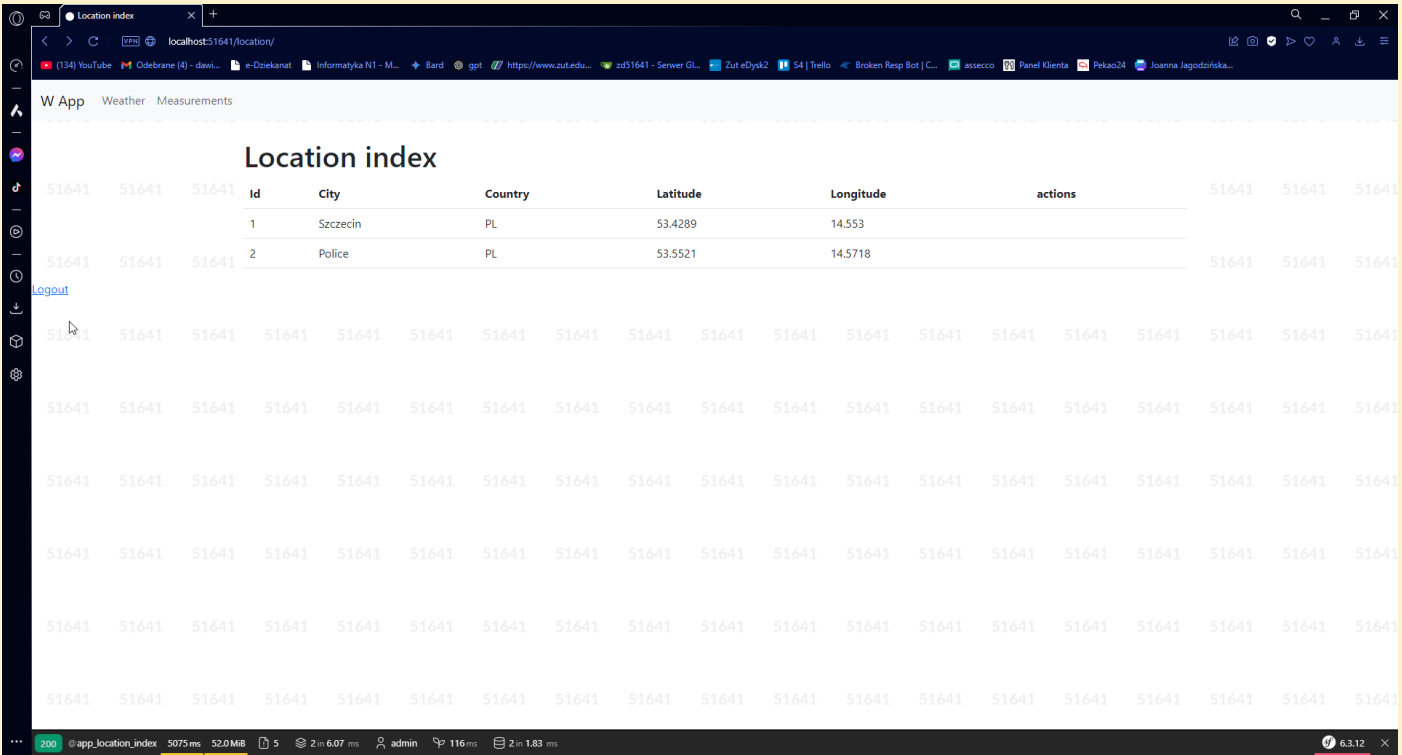
main:	16	16	main:
lazy: true	17	17	lazy: true
provider: app_user_provider	18	18	provider: app_user_provider
http_basic:	>> 19	19 ☐	form_login:
realm: 'My Secured Area'	20	20	login_path: app_login
# activate different ways to authenticate	21	21	check_path: app_login
# https://symfony.com/doc/current/security.html	22	22	logout:
	23	23	path: app_logout

Na koniec należy dodać link do logowania / wylogowywania do pliku szablonu base.html.twig:

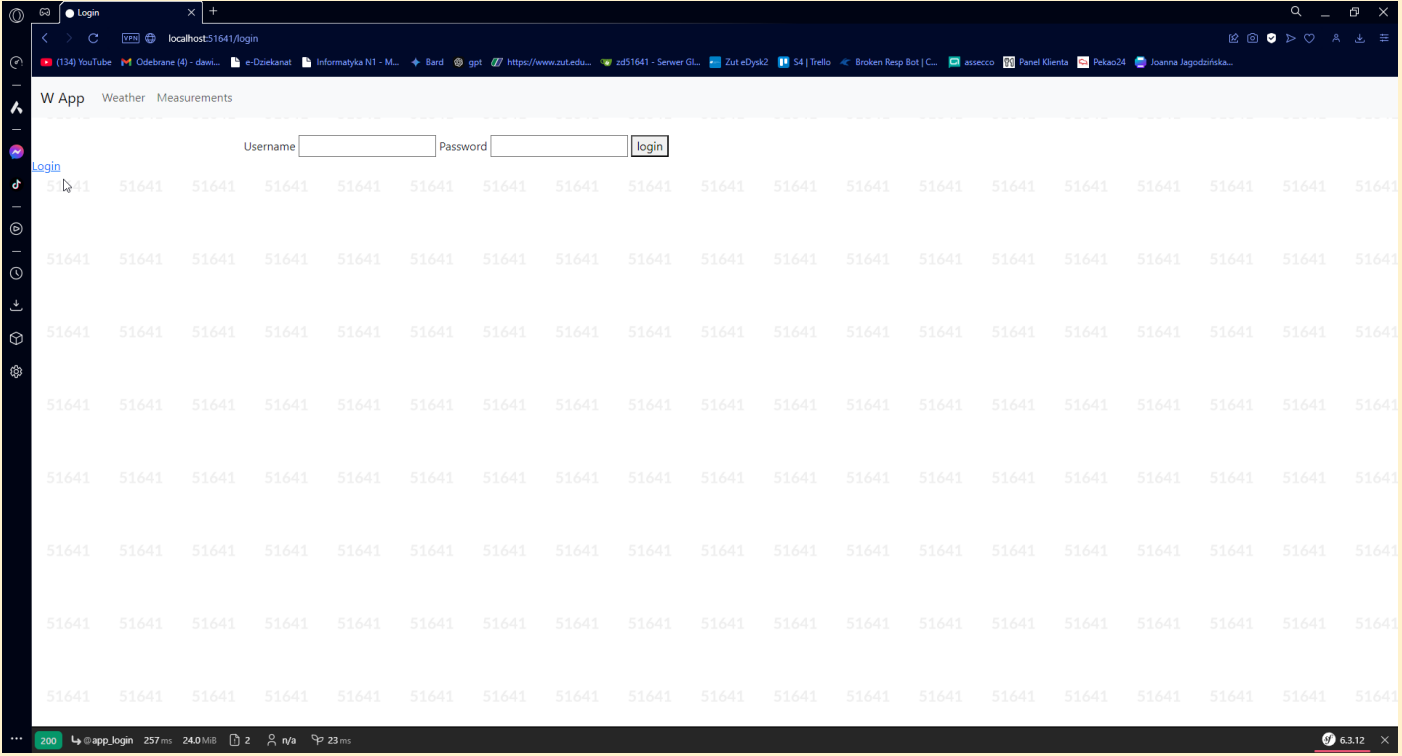
```
{% if is_granted('ROLE_USER') %}
  <a href="{{ path('app_logout') }}">Logout</a>
{% else %}
  <a href="{{ path('app_login') }}">Login</a>
```

```
{% endif %}
```

Wstaw zrzut ekranu strony z widocznym linkiem wylogowywania:



Wstaw zrzut ekranu strony z widocznym linkiem logowania:



Wstaw zrzut fragmentu kodu TWIG odpowiedzialnego za wyświetlanie linku logowania / wylogowywania:

AI2 LAB E – Zielaskowski Dawid – Wersja 1

```
File Edit Selection View Go Run Terminal Help
AI2-lab > pogodynia > templates > base.html.twig
1 <html lang="en">
2 <body>
3 <nav class="navbar navbar-expand-lg navbar-light bg-light">
4 <div class="container-fluid">
5 <div class="collapse navbar-collapse" id="navbarNav">
6 <ul>
7 </ul>
8 </div>
9 </div>
10 </nav>
11 <!-- Main content container -->
12 <div class="container mt-4">
13 {% block body %}
14 <!-- Main content will be injected here -->
15 {% endblock %}
16 </div>
17 {% if is_granted('ROLE_USER') %}
18 <a href="{{ path('app_logout') }}">Logout</a>
19 {% else %}
20 <a href="{{ path('app_login') }}">Login</a>
21 {% endif %}
22 </body>
23 </html>
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Hasher used Symfony\Component\PasswordHasher\Hasher\BcryptHasher
Password hash $2y$13$H4Wu261p1gYE4k1PL0q51N/NCv9KQcP4EycqPM/rQe10kfa
! [NOTE] Self-salting hasher used: the hasher generated its own built-in salt.
[OK] Password hashing succeeded
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynia> cd ..
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab> cd .\pogodynia\
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynia> php .bin/console make:controller Login
created: src\Controller\LoginController.php
created: templates\login\index.html.twig
Success!
Next: Open your new controller class and add some pages!
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynia>
```

Wstaw rzut ekranu konfiguracji security.yaml – fragment dotyczący wylogowywania:

```
File Edit Selection View Go Run Terminal Help
AI2-lab > pogodynia > config > packages > security.yaml
1 security:
2 password_hashers:
3 # https://symfony.com/doc/current/security.html#loading-the-user-the-user-provider
4 providers:
5 # used to reload user from session & other features (e.g. switch_user)
6 app_user_provider:
7 entity:
8 class: App\Entity\User
9 property: username
10 firewalls:
11 dev:
12 pattern: ^/(_(profiler|wdt)|css|images|js)/
13 security: false
14 main:
15 lazy: true
16 provider: app_user_provider
17 form_login:
18 login_path: app_login
19 check_path: app_login
20 logout:
21 path: app_logout
22 # activate different ways to authenticate
23 # https://symfony.com/doc/current/security.html#the-firewall
24 # https://symfony.com/doc/current/security/impersonating_user.html
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Hasher used Symfony\Component\PasswordHasher\Hasher\BcryptHasher
Password hash $2y$13$H4Wu261p1gYE4k1PL0q51N/NCv9KQcP4EycqPM/rQe10kfa
! [NOTE] Self-salting hasher used: the hasher generated its own built-in salt.
[OK] Password hashing succeeded
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynia> cd ..
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab> cd .\pogodynia\
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynia> php .bin/console make:controller Login
created: src\Controller\LoginController.php
created: templates\login\index.html.twig
Success!
Next: Open your new controller class and add some pages!
PS C:\Users\dawid\Desktop\Studia\AI2\AI2-lab\pogodynia>
```

Punkty:

0

1

HIERARCHIA RÓL

W tej sekcji zmodyfikujemy ustawienia `security.yaml` w taki sposób, aby do poszczególnych ról użytkowników (np. `ROLE_ADMIN`) przypisać role zachowań (np. `ROLE_LOCATION_INDEX`):

<pre>access_control: - { path: ^/, roles: PUBLIC_ACCESS } @test: security: password_hashers: # By default, password hashers are resour # important to generate secure password l # are not important, waste resources and</pre>	<pre>30 31 >> 32 33 34 35 36 37 38</pre>	<pre>access_control: - { path: ^/, roles: PUBLIC_ACCESS } role_hierarchy: ROLE_ADMIN: - ROLE_USER - ROLE_LOCATION_EDIT ROLE_USER: - ROLE_LOCATION_INDEX</pre>
--	--	--

W powyższym przykładzie użytkownik o roli `ROLE_ADMIN` może wszystko to co użytkownik o roli `ROLE_USER`, a ponadto może edytować lokalizacje. Użytkownik o roli `ROLE_USER` może jedynie wyświetlać lokalizacje.

Wprowadź zmiany i wejdź na stronę `/location`:

W App Cities Measurements
[Logout](#)

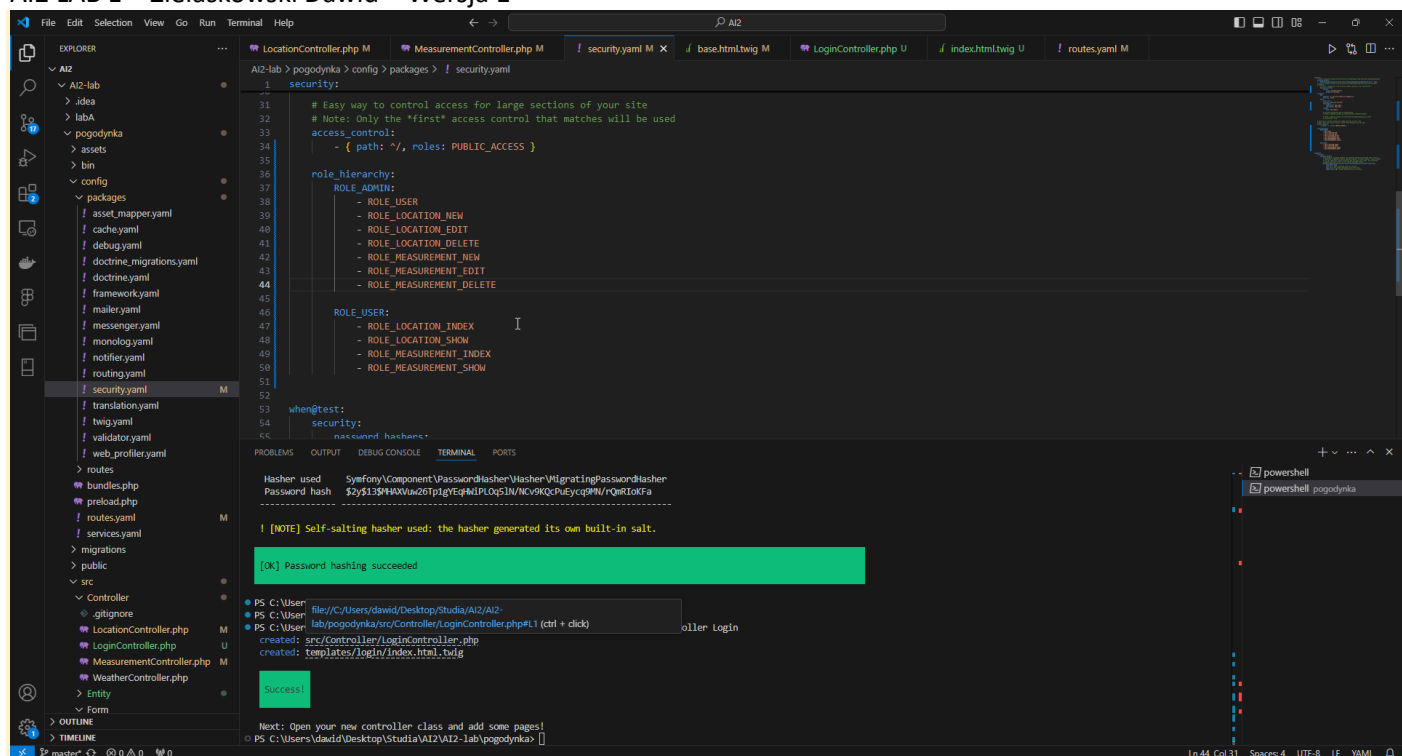
Location index

Id	City	Country	Latitude	Langitude	actions
1	Szczecin	PL	53.4289	14.553	edit weather
2	Police	PL	53.5521	14.5718	edit weather

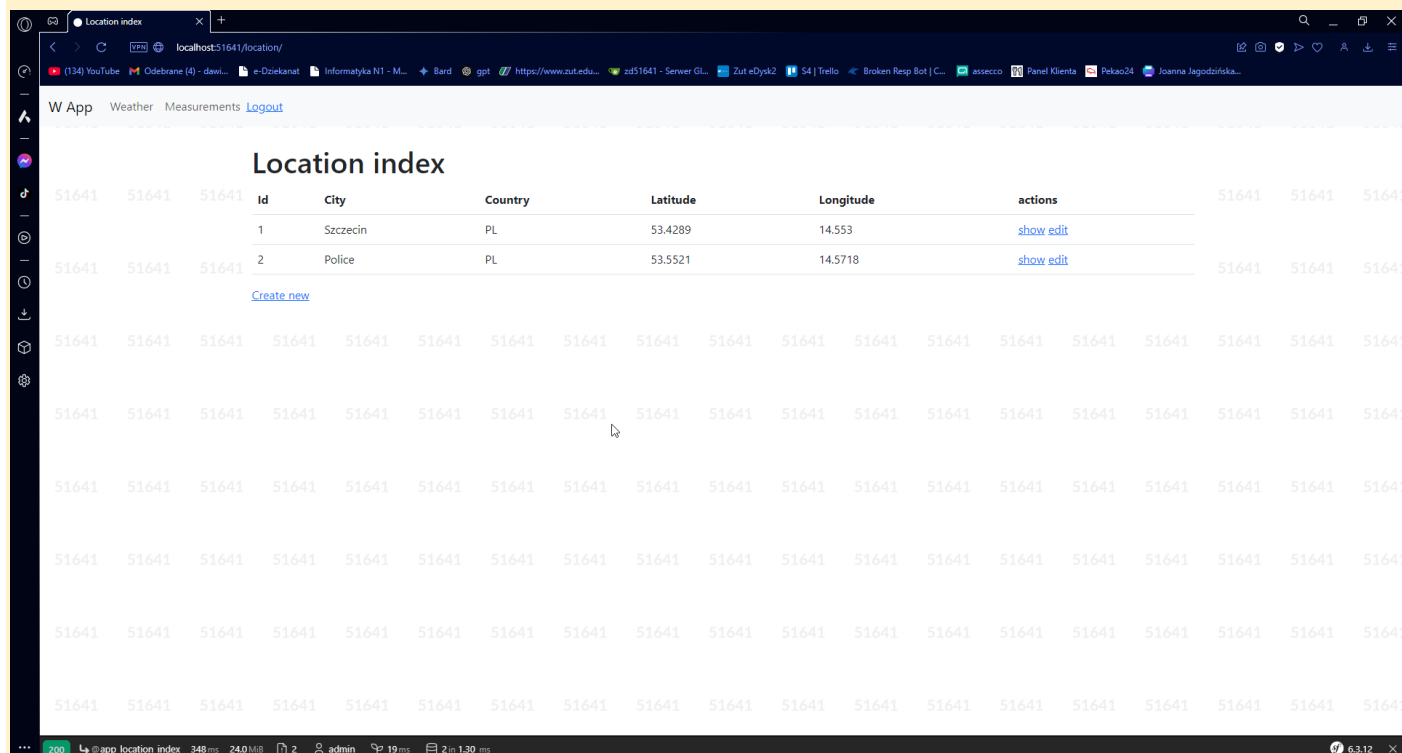
Widoczna jest lista lokalizacji, a przy każdej z nich link do edycji. Nie ma natomiast linku do tworzenia nowej lokalizacji. Wejście na strony pomiarów zakończy się wyświetleniem błędu braku dostępu.

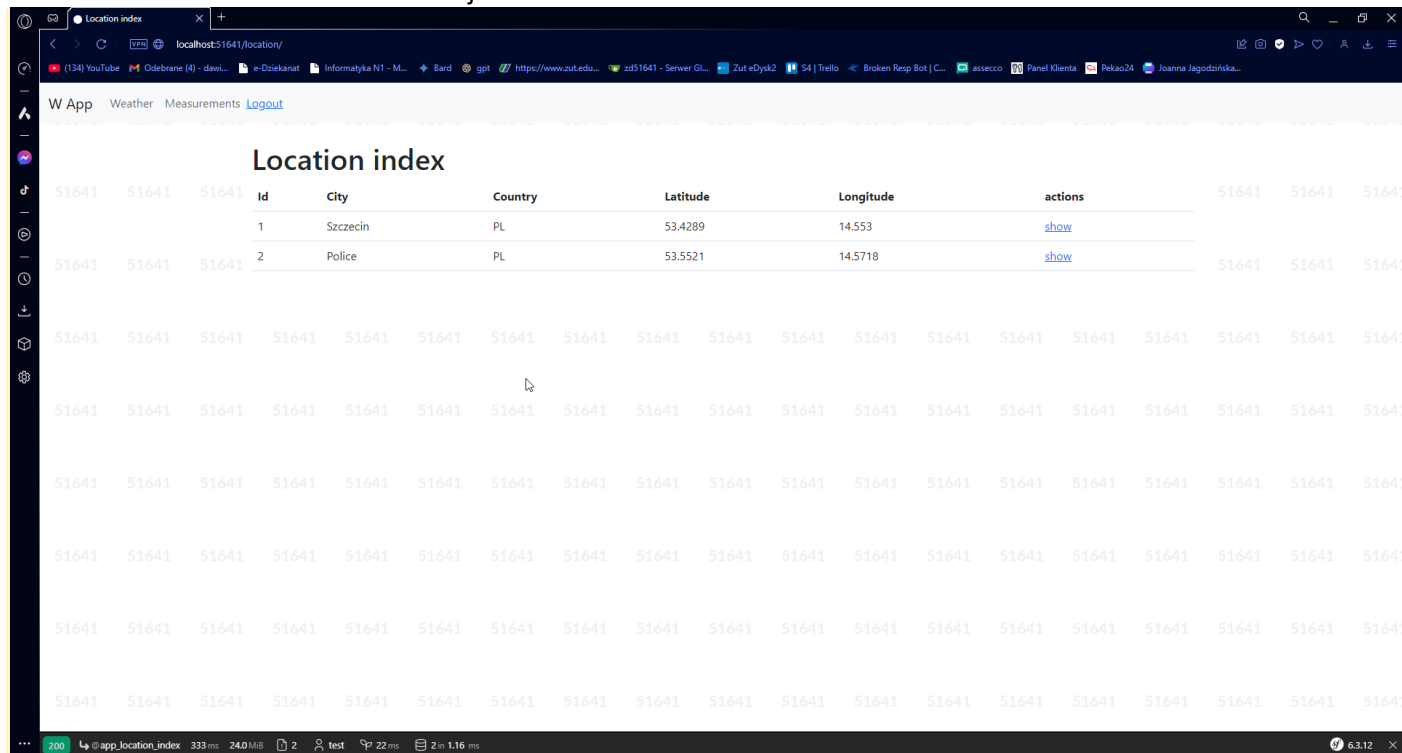
Uzupełnij hierarchię ról dla roli `ROLE_ADMIN` i dla `ROLE_USER`, tak aby obsłużyć wszystkie role obu kontrolerów `Location` i `Measurement`. Wstaw zrzut ekranu odpowiedniego fragmentu `security.yaml`:

AI2 LAB E – Zielaskowski Dawid – Wersja 1



Utwórz nowego użytkownika, z innym zestawem uprawnień (ROLE_USER? Inna, nowa rola użytkownika?). Wstaw zrzut ekranu listy lokalizacji dla pierwszego użytkownika i dla drugiego użytkownika. Upewnij się, że poziom uprawnień jest różny, przez co różnią się dostępne na stronie akcje:





Punkty:

0

1

COMMIT PROJEKTU DO GIT

Zacommituj zmiany. Wyślij zmiany do repozytorium (push). Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-e` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-e` w swoim repozytorium:

<https://github.com/dawid-zielaskowski/ai2-pogodynka/tree/lab-e>

...link, np. <https://github.com/ideaspot-pl/ai2-pogodynka-202310/tree/lab-e...>

PODSUMOWANIE

W kilku zdaniach podsumuj zdobyte podczas tego laboratorium umiejętności.

...podsumowanie...

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.